

SQL to MongoDB Mapping Chart

In addition to the charts that follow, you might want to consider the Frequently Asked Questions section for a selection of common questions about MongoDB.

Terminology and Concepts [↗](#)

The following table presents the various SQL terminology and concepts and the corresponding MongoDB terminology and concepts.

SQL Terms/Concepts	MongoDB Terms/Concepts
database	database
table	collection
row	document or BSON document
column	field
index	index
table joins	<code>\$lookup</code> , embedded documents
primary key	primary key
Specify any unique column or column combination as primary key.	In MongoDB, the primary key is automatically set to the <code>_id</code> field.
aggregation (e.g. group by)	aggregation pipeline See the SQL to Aggregation Mapping Chart.
SELECT INTO NEW_TABLE	<code>\$out</code> See the SQL to Aggregation Mapping Chart.
MERGE INTO TABLE	<code>\$merge</code> See the SQL to Aggregation Mapping Chart.
UNION ALL	<code>\$unionWith</code>
transactions	transactions

SQL	MongoDB Terms/Concepts
Terms/Concepts	TIP For many scenarios, the denormalized data model (embedded documents and arrays) will continue to be optimal for your data and use cases instead of multi-document transactions. That is, for many scenarios, modeling your data appropriately will minimize the need for multi-document transactions.

Executables

The following table presents some database executables and the corresponding MongoDB executables. This table is *not* meant to be exhaustive.

	MongoDB	MySQL	Oracle	Informix	DB2
Database Server	mongod	mysqld	oracle	IDS	DB2 Server
Database Client	mongosh	mysql	sqlplus	DB-Access	DB2 Client

Examples

The following table presents the various SQL statements and the corresponding MongoDB statements. The examples in the table assume the following conditions:

- The SQL examples assume a table named `people`.
- The MongoDB examples assume a collection named `people` that contain documents of the following prototype:

```
{
  _id: ObjectId("509a8fb2f3f4948bd2f983a0"),
  user_id: "abc123",
  age: 55,
  status: 'A'
}
```



Create and Alter

The following table presents the various SQL statements related to table-level actions and the corresponding MongoDB statements.

SQL Schema Statements	MongoDB Schema Statements
CREATE TABLE people (id MEDIUMINT NOT NULL AUTO_INCREMENT,	Implicitly created on first <code>insertOne()</code> or <code>insertMany()</code> operation. The primar key <code>_id</code> is automatically added if <code>_id</code> field is not specified.

SQL Schema Statements

```
user_id Varchar(30),
age Number,
status char(1),
PRIMARY KEY (id)
)
```

```
ALTER TABLE people
ADD join_date DATETIME
```

```
ALTER TABLE people
DROP COLUMN join_date
```

```
CREATE INDEX idx_user_id_asc
ON people(user_id)
```

```
CREATE INDEX
idx_user_id_asc_age_desc
ON people(user_id, age DESC)
```

```
DROP TABLE people
```

MongoDB Schema Statements

```
db.people.insertOne( {
  user_id: "abc123",
  age: 55,
  status: "A"
} )
```

However, you can also explicitly create a collection:

```
db.createCollection("people")
```

Collections do not describe or enforce the structure of its documents; i.e. there is no structural alteration at the collection level.

However, at the document level, `updateMany()` operations can add fields to existing documents using the `$set` operator

```
db.people.updateMany(
{ },
{ $set: { join_date: new Date() } }
)
```

Collections do not describe or enforce the structure of its documents; i.e. there is no structural alteration at the collection level.

However, at the document level, `updateMany()` operations can remove fields from documents using the `$unset` operator

```
db.people.updateMany(
{ },
{ $unset: { "join_date": "" } }
)
```

```
db.people.createIndex( { user_id: 1 } )
```

```
db.people.createIndex( { user_id: 1, age:
-1 } )
```

```
db.people.drop()
```

For more information on the methods and operators used, see:

- `db.collection.insertOne()`
- `db.collection.insertMany()`
- `db.createCollection()`
- `db.collection.updateMany()`
- `db.collection.createIndex()`
- `db.collection.drop()`
- `$set`
- `$unset`

TIP

See also:

- [Databases and Collections in MongoDB](#)
- [Documents](#)
- [Indexes](#)
- [Data Modeling Concepts](#).

Insert

The following table presents the various SQL statements related to inserting records into tables and the corresponding MongoDB statements.

SQL INSERT Statements	MongoDB insertOne() Statements
<pre>INSERT INTO people(user_id, age, status) VALUES ("bcd001", 45, "A")</pre>	<pre>db.people.insertOne({ user_id: "bcd001", age: 45, status: "A" })</pre>

For more information, see `db.collection.insertOne()`.

TIP

See also:

- Insert Documents
- `db.collection.insertMany()`
- Databases and Collections in MongoDB
- Documents

Select

The following table presents the various SQL statements related to reading records from tables and the corresponding MongoDB statements.

NOTE

The `find()` method always includes the `_id` field in the returned documents unless specifically excluded through projection. Some of the SQL queries below may include an `_id` field to reflect this, even if the field is not included in the corresponding `find()` query.

SQL SELECT Statements	MongoDB find() Statements
<pre>SELECT * FROM people</pre>	<pre>db.people.find()</pre>
<pre>SELECT id, user_id, status FROM people</pre>	<pre>db.people.find({ }, { user_id: 1, status: 1 })</pre>
<pre>SELECT user_id, status FROM people</pre>	<pre>db.people.find({ }, { user_id: 1, status: 1, _id: 0 })</pre>
<pre>SELECT * FROM people WHERE status = "A"</pre>	<pre>db.people.find({ status: "A" })</pre>
<pre>SELECT user_id, status FROM people WHERE status = "A"</pre>	<pre>db.people.find({ status: "A" }, { user_id: 1, status: 1, _id: 0 })</pre>
<pre>SELECT *</pre>	<pre>db.people.find(</pre>

SQL SELECT Statements

MongoDB find() Statements

FROM people
WHERE status != "A"

{ status: { \$ne: "A" } }
)

SELECT *
FROM people
WHERE status = "A"
AND age = 50

db.people.find(
{ status: "A",
age: 50 }
)

SELECT *
FROM people
WHERE status = "A"
OR age = 50

db.people.find(
{ \$or: [{ status: "A" } , { age: 50 }] }
)

SELECT *
FROM people
WHERE age > 25

db.people.find(
{ age: { \$gt: 25 } }
)

SELECT *
FROM people
WHERE age < 25

db.people.find(
{ age: { \$lt: 25 } }
)

SELECT *
FROM people
WHERE age > 25
AND age <= 50

db.people.find(
{ age: { \$gt: 25, \$lte: 50 } }
)

SELECT *
FROM people
WHERE user_id like "%bc%"

db.people.find({ user_id: /bc/ })

-or-

db.people.find({ user_id: { \$regex: /bc/ }
})

SELECT *
FROM people
WHERE user_id like "bc%"

db.people.find({ user_id: /^bc/ })

-or-

db.people.find({ user_id: { \$regex: /^bc/
} })

SQL SELECT Statements

MongoDB find() Statements

```
SELECT *
FROM people
WHERE status = "A"
ORDER BY user_id ASC
```

```
db.people.find( { status: "A" } ).sort( {
  user_id: 1 } )
```

```
SELECT *
FROM people
WHERE status = "A"
ORDER BY user_id DESC
```

```
db.people.find( { status: "A" } ).sort( {
  user_id: -1 } )
```

```
SELECT COUNT(*)
FROM people
```

```
db.people.count()
```

or

```
db.people.find().count()
```

```
SELECT COUNT(user_id)
FROM people
```

```
db.people.count( { user_id: { $exists: true
} } )
```

or

```
db.people.find( { user_id: { $exists: true
} } ).count()
```

```
SELECT COUNT(*)
FROM people
WHERE age > 30
```

```
db.people.count( { age: { $gt: 30 } } )
```

or

```
db.people.find( { age: { $gt: 30 } }
).count()
```

```
SELECT DISTINCT(status)
FROM people
```

```
db.people.aggregate( [ { $group : { _id :
"$status" } } ] )
```

or, for distinct value sets that do not exceed the BSON size lir

```
db.people.distinct( "status" )
```

```
SELECT *
FROM people
LIMIT 1
```

```
db.people.findOne()
```

or

SQL SELECT Statements

MongoDB find() Statements

	db.people.find().limit(1)
<pre>SELECT * FROM people LIMIT 5 SKIP 10</pre>	db.people.find().limit(5).skip(10)
<pre>EXPLAIN SELECT * FROM people WHERE status = "A"</pre>	db.people.find({ status: "A" }).explain()

For more information on the methods and operators used, see

- db.collection.find()
- db.collection.distinct()
- db.collection.findOne()
- limit()
- skip()
- explain()
- sort()
- count()
- \$ne
- \$and
- \$or
- \$gt
- \$lt
- \$exists
- \$lte
- \$regex

TIP

See also:

- Query Documents
- Query and Projection Operators

- `mongosh` Methods

Update Records

The following table presents the various SQL statements related to updating existing records in tables and the corresponding MongoDB statements.

SQL Update Statements	MongoDB <code>updateMany()</code> Statements
<pre>UPDATE people SET status = "C" WHERE age > 25</pre>	<pre>db.people.updateMany({ age: { \$gt: 25 } }, { \$set: { status: "C" } })</pre>
<pre>UPDATE people SET age = age + 3 WHERE status = "A"</pre>	<pre>db.people.updateMany({ status: "A" } , { \$inc: { age: 3 } })</pre>

For more information on the method and operators used in the examples, see:

- `db.collection.updateMany()`
- `$gt`
- `$set`
- `$inc`

TIP

See also:

- Update Documents
- Update Operators
- `db.collection.updateOne()`
- `db.collection.replaceOne()`

Delete Records

The following table presents the various SQL statements related to deleting records from tables and the corresponding MongoDB statements.

SQL Delete Statements	MongoDB <code>deleteMany()</code> Statements

SQL Delete Statements

```
DELETE FROM people
WHERE status = "D"
```

```
DELETE FROM people
```

MongoDB deleteMany() Statements

```
db.people.deleteMany( { status: "D" } )
```

```
db.people.deleteMany({})
```

For more information, see `db.collection.deleteMany()`.

TIP

See also:

- Delete Documents
- `db.collection.deleteOne()`

Further Reading

If you are considering migrating your SQL application to MongoDB, download the MongoDB Application Modernization Guide.

The download includes the following resources:

- Presentation on the methodology of data modeling with MongoDB
- White paper covering best practices and considerations for migrating to MongoDB from an RDBMS data model
- Reference MongoDB schema with its RDBMS equivalent
- Application Modernization scorecard